

## Shopping Cart System with Remaining Stock

Create a console-based **Shopping Cart System** in C# using **classes, objects, and an array of objects**. **Requirements:**

1. Create a class named **Product**
2. Add the following fields/properties:
  - Id
  - Name ○ Price ○ RemainingStock
3. Add the following methods:
  - DisplayProduct()
  - **At least one additional method** aside from DisplayProduct() Examples:
    - GetItemTotal(int quantity)
    - HasEnoughStock(int quantity)
    - DeductStock(int quantity) **In Main():**
1. Create an **array of Product objects** as your store menu
2. Display the menu using a loop 3. Ask the user to:
  - Enter product number ○ Enter quantity
4. Validate:
  - Invalid product number ○ Invalid quantity
  - **Non-numeric input** for product number and quantity  
(Use *int.TryParse()* instead of *Convert.ToInt32()* if possible) ○
  - **Stock availability**
  - **Prevent the user from buying more than the available RemainingStock**
  - Display an appropriate message if stock is insufficient (example: **"Not enough stock available."**) ○  
**Handle out-of-stock products** (RemainingStock == 0)
5. If valid: ○ Compute itemTotal = Price \* Quantity ○ Add item to cart
  - Deduct quantity from RemainingStock
6. Prevent **duplicate cart entries** ○ If the same product is selected again, update the existing cart quantity and subtotal instead of adding a new cart row
7. If using a fixed-size cart array: ○ **Prevent adding items if the cart is already full**
  - Display an appropriate message (example: **"Cart is full."**)
8. Allow the user to keep adding items until they choose **N**
9. Display the receipt and **Grand Total**
10. If the **Grand Total is 5000 or more**, apply a **10% discount**
11. Display the **Final Total** after discount
12. Show the **updated remaining stock after checkout**

**Output Requirements:**

Your program should clearly display:

- **Store Menu**
- **Validation messages**
- **Added to cart confirmation**
- **Receipt**
- **Grand Total**
- **Discount amount** (if applicable)
- **Final Total**
- **Updated stock after checkout**

**Additional Requirements:**

1. Create a **verbose flowchart** of your program's process.

Your flowchart should clearly show:

- Start of the program ○ Displaying the menu ○ User selecting a product ○ User entering quantity ○ Validation checks ○ **Stock validation** ○ **Out-of-stock check** ○ Adding to cart
- Updating existing cart item if duplicate
  - Updating remaining stock ○ Checking if cart is full ○ Asking if the user wants to continue ○ Computing grand total ○ Applying discount condition ○ Displaying the final receipt ○ Displaying updated stock ○ End of the program

2. Create a short section titled **"AI Usage in This Project"** and identify:

- Which parts of the project you used AI for (if any)
- Why you used AI in those parts ○ What prompts/questions you asked
- What you changed or improved after using AI

**Submission Requirements:**

Submit the following:

- **C# source code (.cs)**
- **Flowchart** (hand-drawn, digital, or image/PDF)
- **Short AI Usage in This Project explanation**
- **A public Git repository link** containing your complete project files

### Git Repository Requirements:

- Your **Git repository must be public**
- Use the repository naming format:  
**Lastname\_Firstname\_ShoppingCartActivity**
- Your repository must include:
  - Program.cs
  - README.md
  - Flowchart file/image
  - AI Usage explanation (*can be included in README.md*)
- Include at least **3 meaningful commits** that show your development progress Example commits:
  - Initial project setup
  - Added product menu and stock validation
  - Completed cart, discount, and checkout logic
- **Academic Integrity / AI Disclosure:**
- AI tools may be used for **guidance, debugging, or explanation only** • You must **understand and be able to explain** all code that you submit • If AI was used, include:
  - The prompts/questions you asked
  - The parts where AI helped
  - The changes you made after using AI
- Copying code without understanding may affect your grade